

Web Application Development

❖ Advanced CSS(Flexbox, Grid Layout, Media Queries)

AMMAR AHMAD KHAN

What is CSS Flexbox?

- **Flexbox** is short for the **Flexible Box Layout** module.
- Flexbox is a layout method for arranging items in **rows or columns**.
- Flexbox makes it easier to design a flexible **responsive** layout structure, without using **float** or **positioning**.

Flexbox vs Grid

- The CSS **Flexbox** Layout should be used for **one-dimensional** layout, with rows OR columns.
- The CSS **Grid** Layout should be used for **two-dimensional** layout, with rows AND columns.

Flexbox vs Grid



CSS Flexbox Components

- **a Flex Container** - the parent (container) `<div>` element
- **Flex Items** - the items inside the container `<div>`

Div element

What is a <div> Box?

- A <div> (division) is a container element in HTML used to group other HTML elements together.
- Think of a div box as a rectangular container used to organize parts of a webpage.

CSS Flex Container

- The flex container becomes **flexible** by setting the display property to **flex**:

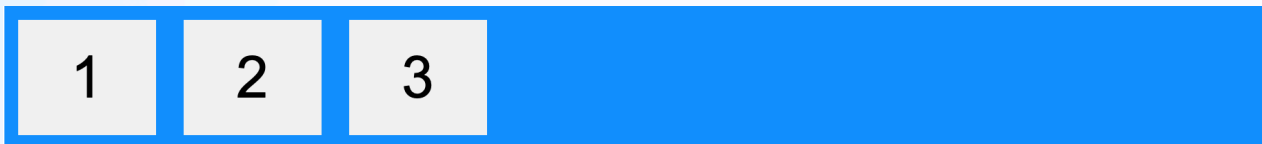
```
.flex-container {  
  display: flex;  
}
```

- The CSS properties we use for the **flex** container are:

- **flex-direction**
- **flex-wrap**
- **flex-flow**
- **justify-content**
- **align-items**
- **align-content**

The CSS flex-direction Property

- The flex-direction property specifies the **display-direction** of flex items in the **flex container**.
- The flex-direction property can have one of the following values:
 - row
 - column
 - row-reverse
 - column-reverse
- The row value is the default value, and it displays the flex items horizontally (from left to right):
 - .flex-container {
 - display: flex;
 - flex-direction: row;
 - }



The CSS flex-wrap Property

- The **flex-wrap** property specifies whether the flex items should **wrap or not**, if there is not enough room for them on one flex line.
- The **flex-wrap** property can have one of the following values:
 - nowrap
 - wrap
 - wrap-reverse
- The **nowrap** value specifies that the flex items will not wrap (this is default):
 - .flex-container {
 - display: flex;
 - flex-wrap: nowrap;
 - }



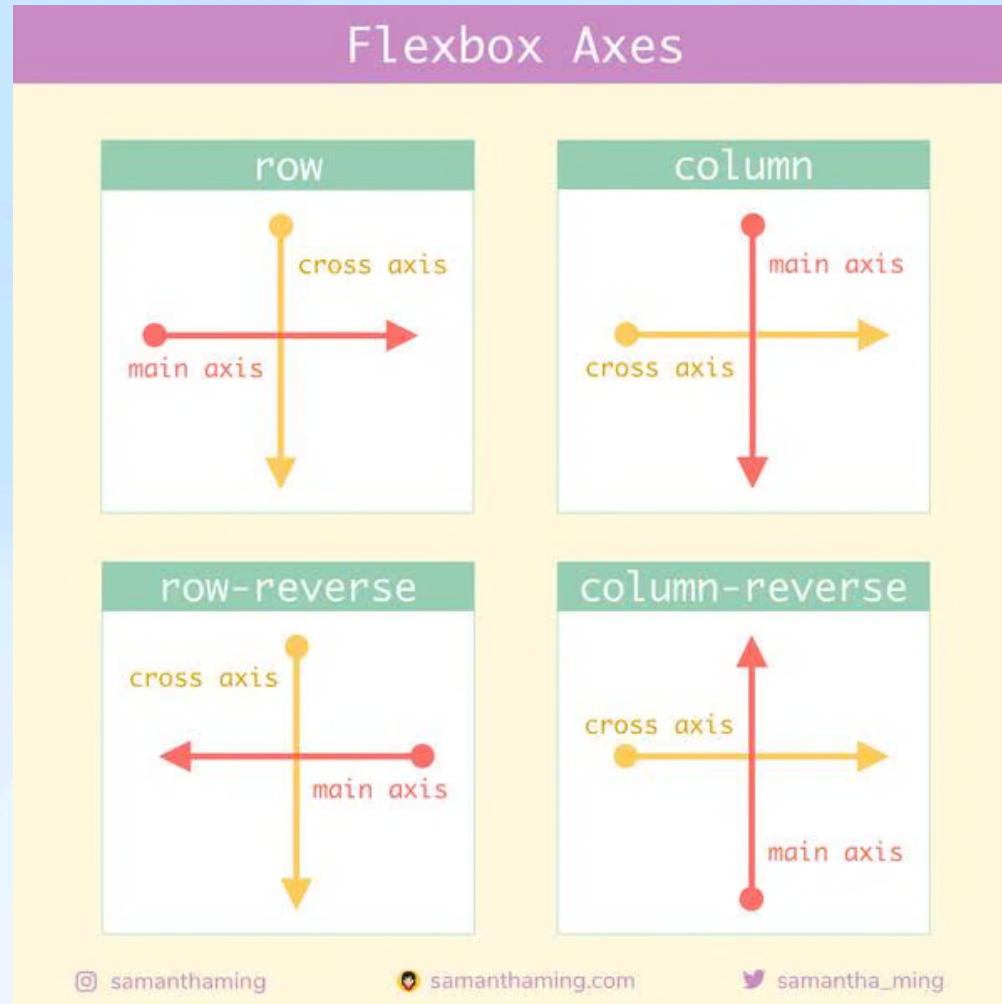
The CSS justify-content Property

- The **justify-content** property is used to align the flex items when they do not use all available space on the **main-axis** (**horizontally**).
- The **justify-content** property can have one of the following values:
 - **center**
 - **flex-start**
 - **flex-end**
 - **space-around**
 - **space-between**
 - **space-evenly**
- The **center** value positions the flex items in the **center** of the container:
 - **.flex-container {**
 - **display: flex;**
 - **justify-content: center;**
 - **}**

•



The CSS justify-content Property



The CSS align-items Property

- The **align-items** property is used to align the flex items when they do not use all available space on the **cross-axis** (**vertically**).
- The **align-items** property can have one of the following values:
 - **center**
 - **flex-start**
 - **flex-end**
 - **stretch**
 - **baseline**
 - **normal**
- The **center** value positions the flex items in the middle of the container:
 - **.flex-container {**
 - **display: flex;**
 - **height: 200px;**
 - **align-items: center;**
 - **}**



CSS Flex Items Properties

- The CSS **properties** we use for **flex items** are:
 - **order**
 - **flex-grow**
 - **flex-shrink**
 - **flex-basis**
 - **flex**
 - **align-self**

CSS Grid Layout

- The **Grid Layout Module** offers a grid-based layout system, with rows and columns.
- The **Grid Layout Module** allows developers to easily create complex web layouts.
- The **Grid Layout Module** makes it easier to design a responsive layout structure, without using float or positioning.
- The **CSS grid** properties are supported in all modern browsers.

CSS Grid Components

- A grid always consists of:
 - a **Grid Container** - the parent (container) `<div>` element
 - **Grid Items** - the items inside the container `<div>`

Grid Container and Grid Items

- A grid layout consists of a parent element (the grid container), with one or more grid items.
- All **direct children** of the grid container automatically become **grid items**.

- `<div class="container">`
- `<div>1</div>`
- `<div>2</div>`
- `<div>3</div>`
- `<div>4</div>`
- `<div>5</div>`
- `<div>6</div>`
- `<div>7</div>`
- `<div>8</div>`
- `</div>`

Display Grid Property

- The **<div>** element becomes a grid container when its display property is set to **grid** or **inline-grid**.
 - `.container {`
 - `display: grid;`
 - `}`

All CSS Grid Properties

Property	Description
<u>align-content</u>	Vertically aligns the whole grid inside the container (when total grid size is smaller than container)
<u>align-items</u>	Aligns content in a grid item along the column axis
<u>align-self</u>	Aligns the content for a specific grid item along the column axis
<u>display</u>	Specifies the display behavior (the type of rendering box) of an element
<u>column-gap</u>	Specifies the gap between the columns
<u>gap</u>	A shorthand property for the <i>row-gap</i> and the <i>column-gap</i> properties
<u>grid</u>	A shorthand property for the <i>grid-template-rows</i> , <i>grid-template-columns</i> , <i>grid-template-areas</i> , <i>grid-auto-rows</i> , <i>grid-auto-columns</i> , and the <i>grid-auto-flow</i> properties
<u>grid-area</u>	Either specifies a name for the grid item, or this property is a shorthand property for the <i>grid-row-start</i> , <i>grid-column-start</i> , <i>grid-row-end</i> , and <i>grid-column-end</i> properties
<u>grid-auto-columns</u>	Specifies a default column size
<u>grid-auto-flow</u>	Specifies how auto-placed items are inserted in the grid
<u>grid-auto-rows</u>	Specifies a default row size
<u>grid-column</u>	A shorthand property for the <i>grid-column-start</i> and the <i>grid-column-end</i> properties

- You can find all the properties at the following link:
https://www.w3schools.com/css/css_grid.asp

CSS Media Queries

- The **@media rule**, introduced in **CSS2**, made it possible to define different style rules for different media types.
- **Media queries** can be used to check many things, such as:
 - **width and height of the viewport**
 - **orientation of the viewport (landscape or portrait)**
 - **resolution**
- Using media queries are a popular technique for delivering a tailored style sheet to desktops, laptops, tablets, and mobile phones (such as iPhone and Android phones).

CSS Media Types

- **Value** **Description**
 - 1. all** Used for all media type devices
 - 2. print** Used for print preview mode
 - 3. screen.** Used for computer screens, tablets smart-phones etc.

CSS Common Media Features

Value	Description
orientation	Orientation of the viewport. Landscape or portrait
max-height	Maximum height of the viewport
min-height	Minimum height of the viewport
height	Height of the viewport (including scrollbar)
max-width	Maximum width of the viewport
min-width	Minimum width of the viewport
width	Width of the viewport (including scrollbar)

Media Query Syntax

- A media query consists of a media type and can contain one or more media features, which resolve to either true or false.

```
@media not | only mediatype and (media feature)
and (media feature) {
    CSS-Code;
}
```

- The mediatype is optional (**if omitted, it will be set to all**). However, if you use **not or only**, you must also specify a mediatype.

Media Queries Simple Examples

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
  background-color: pink;
}

@media screen and (min-width: 480px) {
  body {
    background-color: lightgreen;
  }
}
</style>
</head>
<body>

<h1>Resize the browser window to see the effect!</h1>
<p>The media query will only apply if the media type is screen and the viewport is 480px wide or wider.</p>

</body>
</html>
```

Challenges

